https://pubsonline.informs.org/journal/msom

MANUFACTURING & SERVICE OPERATIONS MANAGEMENT
Vol. 27, No. 6, November–December 2025, pp. 1779–1794
ISSN 1523-4614 (print), ISSN 1526-5498 (online)

Finalist—2023 *M&SOM* Practice-Based Research Competition

Pooling and Boosting for Demand Prediction in Retail: A Transfer Learning Approach

Dazhou Lei,[a] Yongzhi Qi,[b,*] Sheng Liu,[c,*] Dongyang Geng,[b] Jianshen Zhang,[b] Hao Hu,[b] Zuo-Jun Max Shen[d]

[a] School of Economics and Management, Beijing Jiaotong University, Beijing 100044, China; [b] JD.com, Beijing 101111, China; [c] Rotman School of Management, University of Toronto, Toronto, Ontario M5S 3E6, Canada; [d] Faculty of Engineering, Faculty of Business and Economics, University of Hong Kong, Hong Kong, China

*Corresponding authors

Contact: dzlei@bjtu.edu.cn, https://orcid.org/0000-0001-5258-2602 (DL); qiyongzhi1@jd.com (YQ); sheng.liu@rotman.utoronto.ca, https://orcid.org/0000-0003-2365-6013 (SL); gengdongyang@jd.com (DG); zhangjianshen@jd.com (JZ); huhao@jd.com (HH); maxshen@berkeley.edu, https://orcid.org/0000-0003-4538-8312 (Z-JMS)

Received: September 11, 2022
Revised: April 23, 2023; February 7, 2024
Accepted: April 15, 2024
Published Online in Articles in Advance: May 30, 2024

https://doi.org/10.1287/msom.2022.0453

Copyright: © 2024 INFORMS

Abstract. ***Problem definition*:** How should retailers leverage aggregate (category) sales information for individual product demand prediction? Motivated by inventory risk pooling, we develop a new prediction framework that integrates category-product sales information to exploit the benefit of pooling. ***Methodology/results*:** We propose to combine data from different aggregation levels in a transfer learning framework. Our approach treats the top-level sales information as a regularization for fitting the bottom-level prediction model. We characterize the error performance of our model in linear cases and demonstrate the benefit of pooling. Moreover, our approach exploits a natural connection to regularized gradient boosting trees that enable a scalable implementation for large-scale applications. Based on an internal study with JD.com on more than 6,000 weekly observations between 2020 and 2021, we evaluate the out-of-sample forecasting performance of our approach against state-of-the-art benchmarks. The result shows that our approach delivers superior forecasting performance consistently with more than 9% improvement over the benchmark method of JD.com. We further validate its generalizability on a Walmart retail data set and through alternative pooling and prediction methods. ***Managerial implications*:** Using aggregate sales information directly may not help with product demand prediction. Our result highlights the value of transfer learning to demand prediction in retail with both theoretical and empirical support. Based on a conservative estimate of JD.com, the improved forecasts can reduce the operating cost by 0.01–0.29 renminbi (RMB) per sold unit on the retail platform, which implies significant cost savings for the low-margin e-retail business.

History: This paper has been accepted as part of the 2023 *Manufacturing & Service Operations Management* Practice-Based Research Competition.

Funding: This work was supported by the National Natural Science Foundation of China [Grant 71991462].

Supplemental Material: The online appendix is available at https://doi.org/10.1287/msom.2022.0453.

Keywords: retail • demand prediction • transfer learning

1. Introduction

Demand prediction is an imperative task for companies across different industries. In particular, retailers have to base their tactical and operational decisions on short- and long-term demand forecasts. Our collaborator, JD.com, is the leading online retailer in China, with total revenue of $149 billion in 2021 (JD.com 2022). Through its online platform, JD.com sells hundreds of millions of products (stock-keeping units (SKUs)), covering hundreds of product categories and subcategories. Powered by the smart supply chain group, JD.com develops its own demand prediction model that generates forecasts for each SKU on a daily or weekly basis, which then serve as inputs to operational and marketing decisions such as inventory planning and promotion scheduling.

Demand prediction of products in retail is traditionally tackled with time series methods. Given that the products have been on sale for a sufficiently long period, one can use exponential smoothing and autoregressive models to exploit the historical demand pattern such as seasonality and trend.[1] Time series methods often perform well when the feature (covariate) information is limited or the underlying pattern is simple. For more complicated tasks with correlated series and exogenous

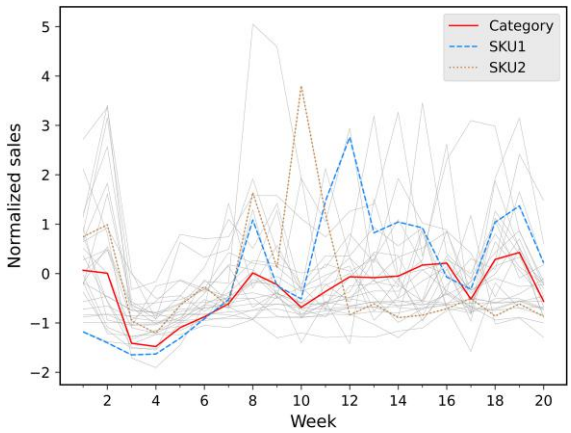
variables, machine learning (ML) algorithms tend to prevail in a data-rich environment. In a recent data competition hosted by the forecasting community (M5 Accuracy competition), all the top-performing methods are ML approaches that achieve significant accuracy improvement over time series and other statistical benchmarks. In particular, gradient boosting machines are shown to be superior among the contested ML approaches (Makridakis et al. 2022a).

One of the most prominent challenges of applying ML in retail demand prediction is the tradeoff between bias and variance. ML models tend to overfit the data and generate prediction results of high variance. Therefore, in addition to devising useful cross-validation strategies, it is common for practitioners to perform a type of data aggregation that fits an aggregate model to SKUs from the same category. For example, one can train a single decision tree model using the data of different cosmetic products together, assuming a shared model structure. This is sometimes referred to as *cross learning*. Data aggregation has a direct variance-reduction effect and is reported to improve the overall prediction performance against training one model for each SKU independently (Cohen et al. 2022a, Makridakis et al. 2022a).

However, a simple data aggregation does not fully exploit the hierarchical structure of data and fails to connect data from different aggregation (grouping) levels. Consider an illustrative two-level setting that consists of one product category (top level) and multiple SKUs (bottom level). One can use data aggregation to fit a single model to all SKU sales observations, but the resulting model disregards the top-level information, that is, category demand. In fact, the sum of the SKU prediction outputs may not behave consistently with the category demand observations. Such inconsistency has stimulated a rich stream of literature on hierarchical forecasting that focuses on harmonizing forecasts from different aggregation levels. However, harmonizing forecasts does not necessarily benefit the bottom-level task (Hyndman et al. 2011).

The classical result from inventory pooling suggests that pooling demands from multiple streams enjoys a lower safety stock and inventory cost, especially when the demand correlation is low and variance is high.[2] Similarly, the category demand data obtained from summing over the SKU demands often exhibit relatively smaller variability compared with SKU data. Figure 1 presents normalized sales data from a subcategory of milk products on JD.com for 20 weeks in 2020, in which the overall category sales curve and two SKU sales curves are highlighted (other SKU sales curves are in gray). It demonstrates that the category sales data are much less volatile than individual SKU sales data. As a result, useful demand pattern information such as seasonality may be better calibrated at the category level; otherwise, it can be masked by the exogenous individual shocks unknown to the decision maker.

Figure 1. (Color online) Sales Curves of a Subcategory of Milk Products



To exploit the pooling effect, one intuitive strategy is to maneuver the category demand data for SKU prediction. For example, a top-down approach is to first generate the category demand forecast and then disaggregate it among SKUs according to some split information (e.g., market share). However, such approaches are often inept in capturing the individual SKU demand pattern and are subject to the bias of the split information.

It remains unclear how to effectively utilize the top-level information for the target bottom-level prediction task. In this paper, we propose a new demand prediction framework for individual SKUs by integrating both category and SKU demand information. Our approach combines ideas from hierarchical forecasting and transfer learning to strike a better balance between bias and variance. Theoretically, we characterize the error bound of our method in linear cases and identify the conditions for its effectiveness. By exploiting a natural connection to gradient boosting tree models, our approach enables an efficient implementation and significantly improves out-of-sample prediction performance over state-of-the-art methods, as demonstrated through internal studies with JD.com and further validation on the Walmart retail data from the M5 Accuracy competition.

1.1. Literature Review

Our paper is related to three streams of literature: demand prediction and retail operations, hierarchical forecasting, and transfer learning.

1.1.1. Demand Prediction and Retail Operations. Demand prediction is critical to retail management and has been a focal research question in retailing (Caro and Gallien 2010, Cohen et al. 2022b). As the evolution of retail progresses, new information from different channels and advanced prediction methods from ML have led to exciting developments in retail demand prediction. On the information side, important features such as weather, promotion, social media, and search trend

have been identified and incorporated into prediction models (Huang et al. 2014, Steinker et al. 2017, Boone et al. 2018, Cui et al. 2018, Fildes et al. 2022, Lei et al. 2023). On the method side, supervised learning models including decision trees and ensemble methods yield promising results for different applications (Ferreira et al. 2016, Guo et al. 2022, Makridakis et al. 2022a, Alley et al. 2023). To avoid overfitting from increased model complexity in high dimensions, clustering approaches and regularization-based methods such as Lasso are shown to be useful (Ma et al. 2016, Baardman et al. 2017, Ban et al. 2019, Hu et al. 2019). Following a similar rationale, our paper provides a new means of regularizing prediction models using aggregate sales information.

Data aggregation is commonly used to mitigate high variability of individual SKU sales observations and the resulting instability of parameter estimates. Specifically, fitting one model for all SKUs across the same department or category is found to be beneficial in several industry applications (Ali et al. 2009, Syntetos et al. 2016). More recently, Cohen et al. (2022a) formalize and study the benefit of data aggregation in generalized linear models. They propose an interesting idea that uses hypothesis testing to identify the right aggregation level for different features. In contrast, we take a different angle than data aggregation and develop a new pooling strategy that integrates category sales observations with SKU sales data using transfer learning.

The tradeoff between estimation stability and model heterogeneity can be exploited by hierarchical Bayesian (HB) approaches. Specifically, HB imposes a common prior distribution on the model parameters when fitting disaggregate data, which enables information sharing across different groups (e.g., SKUs) and exercises a shrinkage effect (Rossi and Allenby 2003, Rossi et al. 2006). HB models and their extensions have shown promising performance for demand modeling of heterogeneous retail stores and consumers (Montgomery 1997, Evgeniou et al. 2007). Our approach does not impose a prior but borrows information from aggregate data to improve estimation stability.

Interestingly, our approach also connects to Adam et al. (2023), where the authors use market share prediction (split information) to correct for product selection bias. Unlike their emphasis on bias correction, our goal is to improve the out-of-sample prediction performance with variance reduction. Furthermore, the idea of data pooling has been exploited in stochastic optimization when multiple subproblems need to be solved simultaneously (Gupta and Kallus 2022). Specifically, they leverage an anchor distribution to improve the decoupled sample average approximation solution for each subproblem. Our approach shares a similar rationale but exploits the sparsity of bias between hierarchical data sets for forecasting tasks.

1.1.2. Hierarchical Forecasting. Forecasting a collection of time series with a hierarchical structure is often referred to as hierarchical forecasting. The main goal of hierarchical forecasting is to generate coherent forecasts such that forecasts at different aggregation levels are consistent. In our setting, coherent forecasts should ensure that the sum of SKU demand forecasts equals the category demand forecast. However, if forecasting models are built independently for different time series, such coherency is often violated (Wickramasuriya et al. 2019).

Two common hierarchical forecasting strategies are "bottom-up" and "top-down" methods. The bottom-up method first generates forecasts at the bottom level and then aggregates them for the higher levels in the hierarchy. In contrast, the top-down method relies on disaggregating the top-level forecasts through the lower-level hierarchy. Despite their popularity, both methods cannot deliver consistently superior results across different data sets (Shlifer and Wolff 1979, Widiarta et al. 2009, Gross and Sohl 2010). More importantly, these two approaches ignore the inherent dependencies among products, including complementarity and substitution effects, causing loss of information (Pennings and Van Dalen 2017). In particular, the top-down method does not recognize the demand pattern at the bottom level and can result in biased forecasts.

A more recent development in hierarchical forecasting is based on optimal combination (or reconciliation), as introduced by Hyndman et al. (2011). This method takes forecasted series at all levels as inputs (called base forecasts) and generates a linear combination of them at different hierarchical levels as new forecasts. Under properly chosen combination weights, the resulting forecasts are unbiased and have minimum variance. However, the forecasting accuracy improvement is not observed across all levels. Similarly, van Erven and Cugliari (2015) propose a reconciliation approach based on minimax optimization, but it does not yield scalable solutions. Lately, Wickramasuriya et al. (2019) develop an improved reconciliation algorithm that overcomes the identification problem of Hyndman et al. (2011). Specifically, they propose to find the optimal combination by minimizing the trace of the covariance matrix of the h-step-ahead reconciled forecast errors. This method is referred to as MinT reconciliation. The MinT approach has shown promising forecasting performance against top-down and bottom-up methods. It has also been extended to generate probabilistic forecasts, account for exogenous information, and provide Bayesian adjustments (Eckert et al. 2021, Taieb et al. 2021, Abolghasemi et al. 2024). A major distinction between our framework and the hierarchical forecasting methods is that our goal is to improve the prediction accuracy of bottom-level tasks (i.e., SKU demand) instead of ensuring coherency. More specifically, we

seek to leverage information from higher-level observations (potentially biased) to enhance bottom-level prediction models.

Given the prevalence of hierarchies in organizations, hierarchical forecasting has received growing attention in operations management, fostering successful applications in emergency service operations (Athanasopoulos et al. 2017), energy consumption prediction (Nystrup et al. 2020), and supply chain management (Mircetic et al. 2022). We hope the prediction framework developed in this work, along with the theoretical and empirical analysis, can lead to further progress in improving forecasting performance for hierarchical data.

1.1.3. Transfer Learning. Our work is also related to transfer learning, of which the objective is to improve the performance of learning algorithms on a target domain (the prediction task of interest) by leveraging knowledge learned from a source domain (Pan and Yang 2010). More specifically, our problem can be categorized as an *inductive transfer learning* problem where the source domain and target domain share the same feature and label spaces, while the prediction tasks are related but different (i.e., the probability distributions of features and labels are different). Transfer learning has been widely applied to natural language processing, computer vision, and robotics (Tan et al. 2018). There are few studies that incorporate the idea of transfer learning into demand prediction, especially in a hierarchical setting. The most closely related is Wellens et al. (2022), where the authors first train representative models using grouped time series (source domain), which are then transferred to lower-level time series. However, they mainly focus on the benefit of computational efficiency while ignoring the pooling effect. In contrast, our transfer learning framework exploits the underlying relationship between category and individual sales observations with pooling.

Tree-based transfer learning models receive relatively less attention than neural networks. The pioneering work of Dai et al. (2007) introduces a boosting tree model (based on adaptive boosting, or AdaBoost) to reweight source data for the target task. Their method is an instance-based transfer approach that seeks to identify "good" data from the source domain to help with the target task. Another strategy is to modify the structure of the tree models when transferring to the target domain, including tree expansion and threshold adaptation techniques (Won Lee and Giraud-Carrier 2007, Segev et al. 2016). Our work contributes to this line of research by developing a gradient boosting tree-based transfer learning model for prediction tasks of a hierarchical structure.

Transfer learning has been relatively underexplored in operations management. Bastani (2021) demonstrates the superior performance of a transfer learning approach for hotel recommendation and medical risk scoring problems. More recently, Bastani et al. (2022) propose a novel metalearning policy for multiproduct dynamic pricing that transfers knowledge across sequential experiments. They exploit the common prior information shared by different products. Our work extends the joint estimator proposed in Bastani (2021) to integrate ideas from hierarchical forecasting and transfer learning. Notably, we generalize their approach from linear models to tree models by establishing a natural connection to gradient boosting.

1.2. Our Contributions

Our work makes several contributions in modeling, analysis, and practical implications.

1. Transfer learning framework with pooling: We develop a new prediction model for forecasting tasks of a hierarchical structure, with a special focus on retail demand forecasting. Motivated by inventory pooling, we propose to combine data from the bottom level and top level in a transfer learning framework. Specifically, we transform the top-level data (aggregate category demand) to form a proxy model in a top-down manner. Then we transfer the trained proxy model to the lower-level data (SKU demand), in which the top-level aggregate information serves as a regularization for the bottom-level SKU demand model. To the best of our knowledge, we are the first to propose transfer learning for demand forecasting with a pooling mechanism. Our model also provides a new lens for hierarchical forecasting through machine learning.

2. Analysis and generalization: We analyze the error performance of our approach in linear models and demonstrate its effectiveness. We show how the error bound depends on the pooling effect and market share distribution. Moreover, we exploit the connection between the joint estimator proposed by Bastani (2021) and gradient boosting trees such that our framework can be implemented in a boosting framework with flexible modeling power and scalable computational performance. With proper regularization, the developed boosting tree framework enjoys (instance-based) out-of-sample performance guarantees. Our approach can also be generalized to pooling across time and neural network models.

3. Empirical study with a large online retailer: We conduct experiments with JD.com on two representative subcategories of more than 6,000 weekly SKU sales observations. The out-of-sample performance evaluation results demonstrate the dominating performance of the proposed approach. Compared with JD.com's benchmark and other state-of-the-art methods, our approach yields 9%–14% lower forecasting errors across different metrics. This improvement can translate to cost savings of 0.01–0.29 renminbi (RMB) per sold unit based on a conservative estimate from JD.com

(which equals millions of dollars of annual cost savings). We further validate its generalizability on sales data of a different aggregation level and from the M5 Accuracy competition.

The remainder of the paper is organized as follows. Section 2 introduces the model and performs error analysis. Section 3 presents the model evaluation results with JD.com and provides operational implications. Section 4 reports additional evaluation results on a public data set. Section 5 concludes the paper and discusses future research directions.

2. Pooling and Boosting (PAB) Method

Let $[m] = \{1, \ldots, m\}$ denote the set of SKUs belonging to the same category and $[n] = \{1, \ldots, n\}$ denote the set of observations (e.g., weekly sales observations). The feature set is denoted by $[d] = \{1, \ldots, d\}$. For product $i \in [m]$, we have access to the historical sales vector $Y_i = (y_{i1}, \ldots, y_{in})^\top \in \mathbb{R}^n$ and feature matrix $\boldsymbol{X}_i = (\mathbf{x}_{i1}, \ldots, \mathbf{x}_{in})^\top \in \mathbb{R}^{n \times d}$, where $\mathbf{x}_{ij} = (x_{ij}^1, \ldots, x_{ij}^d)^\top \in \mathbb{R}^d$ for $j \in [n]$. Stacking the product sales and features together, we define $Y = (Y_1^\top, \ldots, Y_m^\top)^\top \in \mathbb{R}^{nm}$ and $\boldsymbol{X} = (\boldsymbol{X}_1^\top, \ldots, \boldsymbol{X}_m^\top)^\top \in \mathbb{R}^{nm \times d}$. Moreover, by pooling product sales observations according to their category membership, we can build the category (or subcategory) sales vector $Y_c = (y_{c1}, \ldots, y_{cn})^\top \in \mathbb{R}^n$, where $y_{cj} = \sum_{i \in [m]} y_{ij}$ for $j \in [n]$. Following the standard practice, the feature matrix is standardized for each feature, that is, $(1/nm)\boldsymbol{X}^\top \boldsymbol{X}$ has diagonal elements equal to one.[3] Our goal is to predict future (and potentially new) product sales accurately given known feature information, which can be cast as a supervised learning problem or regression problem. Our proposed PAB method leverages pooled category sales data and individual SKU sales data to strike a better balance between bias and variance.

2.1. Main Idea

Instead of fitting regression models to individual product sales data directly, PAB first builds a *proxy* data set by splitting the category sales data according to a prespecified split vector. The proxy data enjoy the benefit of pooling when the category sales exhibit a relatively smaller variability compared with individual sales. Next, PAB fits a regression model to the proxy data, which serves as a "regularization" when further fitting a regression model to individual product sales data. During fitting, PAB adopts a gradient boosting procedure to connect the two fitted regression models and enhance their prediction performance.

We first illustrate our method under the linear model framework, in which boosting is not involved. With linear assumptions, we demonstrate the value of pooling in demand prediction through error bound analysis. We then extend linear models to gradient boosting trees that admit better modeling flexibility and prediction power.

2.2. Linear Models

Under the linear model assumption, the demand for product i follows (for ease of presentation, we drop the observation subscript j in this section whenever no confusion may arise)

$$y_i = \mathbf{x}_i^\top \beta + \epsilon_i, \quad i \in [m].$$

Here, $\beta \in \mathbb{R}^d$ is the unknown regression coefficient vector that needs to be estimated. ϵ_i is the random noise that follows an independent subgaussian distribution with parameter σ. However, we do not assume ϵ_i is independent across products, that is, ϵ_i can be correlated with $\epsilon_{i'}$ for $i, i' \in [m]$. This is a pronounced feature of practical sales observations of products under the same category, where substitution patterns can drive negative correlations among products (or positive correlations caused by external shocks that affect the category sales as a whole). Here, we assume the model is estimated at the category level, that is, different products share the same coefficient vector. This aggregation strategy is shown to deliver superior prediction performance in retail applications (Makridakis et al. 2022a). Nevertheless, our results can be extended to the case where the model is set up for each product separately.

Accordingly, the aggregate category demand follows

$$y_c = \sum_{i \in [m]} \mathbf{x}_i^\top \beta + \sum_{i \in [m]} \epsilon_i.$$

To model the relationship between category demand and SKU demands, we assume there is an underlying true split vector $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_m)^\top \in \mathbb{R}_+^m$ with $\sum_{i \in [m]} \omega_i = 1$ such that

$$\mathbf{x}_i^\top \beta = \omega_i \sum_{i \in [m]} \mathbf{x}_i^\top \beta = \omega_i \left(\sum_{i \in [m]} \mathbf{x}_i \right)^\top \beta, \quad i \in [m]. \tag{1}$$

This split information reflects that each product's market share is relatively stable regardless of the overall category sales. Whereas the market can be volatile, the products' relative attractiveness does not change drastically.

The true $\boldsymbol{\omega}$ is unknown to us, but we can obtain a proxy (and potentially biased) split vector $\tilde{\boldsymbol{\omega}}$ learned from market research or data analysis. Then we apply this proxy split vector to the category demand and construct *proxy demand data* as

$$\tilde{y}_i = \tilde{\omega}_i \left(\sum_{i \in [m]} \mathbf{x}_i \right)^\top \beta + \tilde{\omega}_i \sum_{i \in [m]} \epsilon_i, \quad i \in [m].$$

By the assumption made in Equation (1), we can rewrite the above equation as

$$\tilde{y}_i = \frac{\tilde{\omega}_i}{\omega_i} \mathbf{x}_i^\top \beta + \tilde{\omega}_i \sum_{i \in [m]} \epsilon_i = \mathbf{x}_i^\top \left(\frac{\tilde{\omega}_i}{\omega_i} \beta \right) + \tilde{\omega}_i \sum_{i \in [m]} \epsilon_i, \quad i \in [m]. \tag{2}$$

Compared with the original demand data y_i, the proxy demand data $\tilde{y}_i$ is potentially biased for estimating β

due to the mismatch between $\tilde{\omega}$ and ω. On the other hand, it can enjoy the benefit of pooling from the variability reduction of $\sum_{i\in[m]}\epsilon_i$. Let $\tilde{\epsilon}=\sum_{i\in[m]}\epsilon_i$, and it is subgaussian with parameter $\tilde{\sigma}$. In the special case where $\epsilon_i \sim N(0,\sigma^2)$ and the pairwise correlation of ϵ_i's is ρ, we have $\tilde{\sigma}^2 = m\sigma^2 + m(m-1)\rho\sigma^2$. With a negative value of ρ, the variance of $\tilde{\omega}_i\tilde{\epsilon}$ tends to be much smaller than σ^2. For instance, if $\tilde{\omega}_i = 1/m$, $Var(\tilde{\omega}_i\tilde{\epsilon}) < \sigma^2/m$ whenever $\rho < 0$. We will exploit this pooling benefit in the proposed regression method described below. Note, however, that we do not impose any assumptions on the correlation structure of product sales.

Given the original and proxy sales data, we apply the joint estimator developed in Bastani (2021), in which the original sales data y_i are treated as the *gold data* in contrast to the proxy data $\tilde{y}_i$. The two data sets share the same feature matrix, but the underlying regression coefficients are different. While the coefficient of interest β can be identified from fitting the gold data, the coefficient governing the proxy data generating process is $\tilde{\omega}_i\beta/\omega_i \neq \beta$. Therefore, running a regression on the proxy data will incur a bias of $\Delta_i = (1-\tilde{\omega}_i/\omega_i)\beta$.

We suppose the bias is small with the assumption that the decision maker (retailer) can make a good estimate of the split vector, that is, $\tilde{\omega} \approx \omega$. More specifically, we assume $(\tilde{\omega}_i/\omega_i)\beta \approx \beta$ for $i \in [m]$ and $\Delta_i \approx \Delta$ is sparse (or approximately sparse), that is, only a small number of elements of Δ are nonzero. Mathematically, we require $\|\Delta\|_0 = s < d$. In practice, when the regression coefficients are heterogeneous on the scale (e.g., β is already approximately sparse), the term $(1-\tilde{\omega}_i/\omega_i)$ exercises a scaling-down effect, so the smaller coefficients are shrunk toward zero and make a negligible contribution to Δ. Moreover, by construction, the proxy demands often follow similar temporal patterns as the original demands, which suggests the bias for the temporal features can be small. We also note that the number of features in retail forecasting can be large, in which sparsity is desired to improve the prediction performance (Cui et al. 2018, Fildes et al. 2022). Although sparsity is not a prerequisite to the analysis below, the joint estimator of Bastani (2021) is most effective when a strong sparsity is present, that is, $s \ll d$.

The joint estimator works with the gold and proxy data as follows. In the first step, we run an ordinary least squares (OLS) regression on the proxy data and obtain coefficient estimate $\hat{\beta}_{proxy}$ as

$$\hat{\beta}_{proxy} = \arg\min_{\beta}\left\{\frac{1}{nm}\sum_{i\in[m]}\|\tilde{Y}_i - \boldsymbol{X}_i\beta\|_2^2\right\}, \tag{3}$$

where $\tilde{Y}_i = (\tilde{y}_{i1},\ldots,\tilde{y}_{in})^\top$ is the proxy demand vector of product i. In the second step, a Lasso-type of regression is run on the gold data (original sales data) with a regularization term on the difference between the estimated coefficient and $\hat{\beta}_{proxy}$:

$$\hat{\beta}_{joint} = \arg\min_{\beta}\left\{\frac{1}{nm}\sum_{i\in[m]}\|Y_i - \boldsymbol{X}_i\beta\|_2^2 + \lambda\|\beta - \hat{\beta}_{proxy}\|_1\right\}, \tag{4}$$

where $\lambda > 0$ is a regularization parameter that controls the relative weight put on the proxy data.[4] One can also write the above regression equation in a more compact form using Y and $\boldsymbol{X}$, that is, with $\|Y - \boldsymbol{X}\beta\|_2^2 = \sum_{i\in[m]}\|Y_i - \boldsymbol{X}_i\beta\|_2^2$. The second-step regression leverages the ℓ_1-norm regularization to exploit the sparse structure of Δ. As such, the proxy demand from category sales data performs a regularization to the product demand model. We present below the main theoretical result for this estimator in our problem.

To show the result, we make the following assumptions.

Assumption 1. (a) *The feature sample covariance matrix* $\Sigma = (1/nm)\boldsymbol{X}^T\boldsymbol{X}$ *is positive definite with the smallest eigenvalue* $\psi > 0$. (b) $\|\beta\|_1 \le b$ *for some constant* $b \in \mathbb{R}$.

Assumption 1 is standard in statistics for identification. Then it follows that the estimation error of $\hat{\beta}_{joint}$ can be bounded when it is further truncated.

Theorem 1. *The estimation error of the joint estimator satisfies*

$$\mathbb{E}[\|\hat{\beta}^{\diamond}_{joint} - \beta\|_1] = O\left(\max\left\{\frac{s\sigma}{\sqrt{n}}\log(dn), \frac{sd\tilde{\sigma}\sqrt{\sum_{i\in[m]}\tilde{\omega}_i^2}}{\sqrt{mn}}\log(dmn)\right\}\right),$$

where $\hat{\beta}^{\diamond}_{joint} = \hat{\beta}_{joint}$ *if* $\|\hat{\beta}_{joint}\|_1 \le 2b$ *and* 0 *otherwise.*

The above result can be derived following theorem 5 and corollary 1 of Bastani (2021) with some notable differences. First, because the product sales can be correlated, both the gold error and proxy error terms are not independently distributed (while independent across observations). To overcome this, we stack the product sales data properly over time so we can apply the concentration bound to the aggregated parts that are correlated (but still subgaussian). In the meantime, the independence across observations is leveraged to sharpen the bound. Second, in our setting, the proxy data share the same feature matrix with the gold data, so we can drop the compatibility condition of Bastani (2021) as long as Assumption 1 is satisfied.

The error bound presented in Theorem 1 is a worst-case bound because it holds for any combination of the problem parameters $(\boldsymbol{X},\beta,\Delta)$. The bound illustrates how the estimation error is shaped by pooling and the proxy split vector. Specifically, the second term of the

error bound depends on $\tilde{\sigma}$ and $\sqrt{\sum_{i\in[m]}\tilde{\omega}_i^2}$. Intuitively, this term is decreasing in the variability reduction from pooling, that is, $\tilde{\sigma}$. We observe that the second term tends to be greater than the first term when $d > \sqrt{m}$, which applies to many retail applications. For example, Wellens et al. (2024) constructed 104 features for their retail sales forecasting model, including lagged sales, holidays, special events, weather, and price variables. Additionally, in renewable energy generation prediction problems, the candidate features encode physical characteristics (e.g., meteorologically relevant factors) and can be of high dimension, whereas m typically refers to the number of spatial units, which is relatively small (Ye et al. 2023). Therefore, the decrease of $\tilde{\sigma}$ from pooling can lead to a smaller estimation error of the joint estimator.

The proxy split vector affects both terms of the error bound: the bias of the proxy split vector can inflate s, whereas the second term involves $\sqrt{\sum_{i\in[m]}\tilde{\omega}_i^2}$. We can check that the second term has a lower bound of $(sd\tilde{\sigma}/m\sqrt{n})\log(dmn)$, which is achieved when $\tilde{\omega}_i = 1/m$ for all $i \in [m]$. This implies that the error of the designed joint estimator becomes smaller when the market share is more evenly distributed, and demand substitution (pooling effect) is more pronounced. Note that, if the underlying true $\omega_i \neq 1/m$ for all $i \in [m]$, setting $\tilde{\omega}_i = 1/m$ may cause Δ and s to increase, resulting in a higher estimation error. We provide numerical results on synthetic data sets that validate our analysis and bound performance in Online Appendix EC.2. Theoretically, having $\tilde{\omega} \approx \omega$ strengthens the sparsity condition and leads to improved error performance. Empirically, we find that choosing a proxy split vector that is close to the true market share yields preferable out-of-sample prediction performance. In practice, the empirical estimates of the market share can be formed based on (i) moving average—the most recent $k \in \{4, 8, 12\}$ observed sales are used to estimate the market share, that is, the ratio of individual sales to the aggregate sales, which is taken as the proxy split vector; (ii) fixed-time average—instead of using the most recent observations, the observed sales from the last month/season are used to estimate the market share and the proxy split vector, which is not updated within the same month/season. These methods may be improved with more customized market-share models such as attraction and multinomial logit (MNL) models to incorporate product characteristics (Cooper 1993, Nevo 2000).

Theorem 1 also sheds light on the relative advantage of the joint estimator over the original OLS estimator. Based on the analysis of Bastani (2021), a simple OLS estimator applied to the original demand data causes an error at least in the order of $d\sigma/\sqrt{nm}$ (a lower bound). When m is not so large, both terms of the error bound are lower than $d\sigma/\sqrt{nm}$ if $s \ll d$ and $\tilde{\sigma}\sqrt{\sum_{i\in[m]}\tilde{\omega}_i^2} \ll \sigma$. In this case, the relative improvement of the joint estimator over the OLS estimator is most pronounced. Such improvement echoes the insights from inventory pooling that suggest the benefit of pooling is more significant when the individual demand variability is high but the demand correlation is low. Note that we drop the lower-order terms in the bound, and the error will not be zero even when $s = 0$ (e.g., $\hat{\omega} = \omega$).

Remark 1. When mapped to the transfer learning framework, the joint estimator presented above can be classified as an inductive transfer learning approach, in which the source domain (proxy data) and the target domain (gold data) share the same feature space but different target distributions. However, the domain task and target task are related because they characterize similar demand patterns.

Remark 2. The tradeoff between variability reduction and bias of the proxy split vector depends on the product category structure, including the market share and correlation relationships. With more positively correlated product sales, the variability reduction is less pronounced, whereas the induced bias of the split vector can be relatively small (under empirical estimation methods such as moving averages). This is because the ratio of the individual product sales to the aggregate sales is relatively stable when sales are all positively correlated.

Remark 3. The error bound on $\hat{\beta}^{\diamond}_{joint}$ can translate to bounds on the in-sample and out-of-sample risk (prediction error). Because $\hat{\beta}^{\diamond}_{joint}$ and β are bounded, the in-sample and out-of-sample risks (with squared error loss) are in the order of $\sqrt{\log d}$ (Tibshirani and Wasserman 2017).

Despite their effectiveness in some applications, linear models and the corresponding estimators often lack the flexibility to model more complicated and nonlinear relationships between demand and product features. In practice, more advanced machine learning models such as tree-based models are popular and often enjoy better prediction performance (Ferreira et al. 2016). Building on the joint estimator, we next present the PAB method that extends the idea of pooling to gradient boosting trees.

2.3. Gradient Boosting and PAB

Boosting methods are among the most significant ensemble methods that combine multiple base learners to generate a more powerful prediction model. In general, boosting tries to find an ensemble model $F(x) = \sum_{\ell\in[t]}\alpha_\ell h_\ell(x)$ that consists of base functions $h_\ell(x)$ for the ensemble weights α_ℓ, $\ell \in [t]$, to minimize the empirical risk with respect to a loss function $L(\cdot,\cdot) : \mathbb{R}\times\mathbb{R} \to \mathbb{R}$

(we suppress the product index i in this section for brevity), which is

$$\frac{1}{n}\sum_{j\in[n]} L\left(y_j, \sum_{\ell\in[t]} \alpha_\ell h_\ell(\mathbf{x}_j)\right). \tag{5}$$

The decision variables are base functions from a specific functional space $\mathcal{F}$. When imposing structural restrictions on $\mathcal{F}$, the problem boils down to searching for the optimal set of parameters characterizing $h_\ell(\mathbf{x})$, $\ell \in [t]$, that minimize (5).

2.3.1. Gradient Boosting Trees. Tree-based models (e.g., regression trees) are popular choices of base learners for boosting. One can parameterize a tree model by a partition Ψ of the feature space and a predictive rule γ that associates a prediction value with each region of the partition. The partition corresponds to the set of leaf nodes in a tree. Then a boosted tree model follows by minimizing the empirical loss defined as

$$\min_{\{\Psi_\ell,\gamma_\ell\}_{\ell\in[t]}} \sum_{j\in[n]} L\left(y_j, \sum_{\ell\in[t]} \alpha_\ell T(\mathbf{x}_j;\Psi_\ell,\gamma_\ell)\right), \tag{6}$$

where $T(\mathbf{x};\Psi_\ell,\gamma_\ell)$ represents a tree model parameterized by Ψ_ℓ and γ_ℓ. Finding a global optimal solution to (6) is challenging due to the combinatorial tree structure, so a common strategy is to find an approximate solution using a greedy algorithm (also known as additive training). Under this strategy, the base tree models are built sequentially, and the models that have already been added in the ensemble are fixed when optimizing over the newly added one. More specifically, in the k-th iteration of the algorithm, a tree is trained such that the difference between its prediction and the negative gradient of the loss function evaluated at $\sum_{\ell=1}^{k-1}\alpha_\ell T(\mathbf{x}_j;\Psi_\ell,\gamma_\ell)$ is minimal:

$$\min_{\Psi_k,\gamma_k} \sum_{j\in[n]} \left(-g_j\left(\sum_{\ell=1}^{k-1}\alpha_\ell T(\mathbf{x}_j;\Psi_\ell,\gamma_\ell)\right) - T(\mathbf{x}_j;\Psi_k,\gamma_k)\right)^2, \tag{7}$$

where $g_j(f(\mathbf{x}_j)) = \partial L(y_j, f(\mathbf{x}_j))/\partial f(\mathbf{x}_j)$. The resulting boosted trees are referred to as *gradient boosting trees* (other boosting methods that adopt different objective functions than (7) are omitted here). The weight α_k of the k-th tree is also called the shrinkage parameter, which shrinks the prediction value of the newly added tree (we assume $0 \le \alpha_k \le 1$). It is often assumed that $\alpha_k = \alpha$ for all k, which is treated as a tuning parameter.

Gradient boosting trees have been implemented efficiently using various distributed computing frameworks, which are available in open-source packages such as XGBoost and LightGBM (Chen and Guestrin 2016, Ke et al. 2017). Successful applications of XGBoost and LightGBM have been acclaimed by major data competition platforms and leading industry companies such as Kaggle and Uber (Wang et al. 2019). We refer interested readers to a more complete introduction to gradient boosting trees in Hastie et al. (2009).

2.3.2. Marrying the Joint Estimator with Gradient Boosting Trees. We present our PAB method under the framework of gradient boosting trees, which yields a natural connection to the joint estimator studied in Section 2.2. It relies on an important interpretation (and reformulation) of the joint estimator: the second stage of the regression, as done in Equation (4), is equivalent to finding $\hat{\delta}$ that satisfies (Bastani 2021):

$$\hat{\delta} = \arg\min_{\delta}\left\{\frac{1}{nm}\sum_{i\in[m]}\|Y_i - \boldsymbol{X}_i(\hat{\beta}_{proxy}+\delta)\|_2^2 + \lambda\|\delta\|_1\right\}, \tag{8}$$

which leads to $\hat{\beta}_{joint} = \hat{\beta}_{proxy} + \hat{\delta}$. This reformulation reflects that the second-stage regression aims to correct for the bias of the proxy estimator by estimating $\hat{\delta}$ from the gold data.

Using gradient boosting trees as the prediction model, we can write the second-stage problem analogous to Equation (8) as

$$\min_{\{\Psi_\ell,\gamma_\ell\}_{\ell\in[t]/[t_1]}} \sum_{i\in[m]}\sum_{j\in[n]} L\left(y_{ij}, \sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x}_{ij};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy}) + \sum_{\ell\in[t]/[t_1]}\alpha_\ell T(\mathbf{x}_{ij};\Psi_\ell,\gamma_\ell)\right) + \sum_{\ell\in[t]/[t_1]}\Omega(\Psi_\ell,\gamma_\ell), \tag{9}$$

where the ensemble $\sum_{\ell\in[t_1]}\alpha_\ell T(x_{ij};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy})$ consists of t_1 ($0 < t_1 < t$) trees built from the proxy data (as in the first stage of the joint estimator):

$$\{\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy}\}_{\ell\in[t_1]} = \arg\min_{\{\Psi_\ell,\gamma_\ell\}_{\ell\in[t_1]}} \sum_{i\in[m]}\sum_{j\in[n]} L\left(\tilde{y}_{ij}, \sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x}_{ij};R_\ell,\gamma_\ell)\right), \tag{10}$$

and $\Omega(\Psi_\ell,\gamma_\ell)$ is the regularization function of tree $\ell \in [t]/[t_1] = \{t_1+1,\ldots,t\}$. Here, $\sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x}_{ij};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy})$ serves as the proxy prediction model, while $\sum_{\ell\in[t]/[t_1]}\alpha_\ell T(\mathbf{x}_{ij};\Psi_\ell,\gamma_\ell)$ is an "additive" ensemble model that accounts for the bias of the proxy prediction model. In a different application, a two-stage fitting process is also used to estimate nonparametric tree-based choice models, which helps reduce the possibility of overfitting (Chen and Mišić 2022).

Note that the regularization term $\Omega(\Psi_\ell,\gamma_\ell)$ encourages a sparse structure by penalizing the tree model complexity. We consider both the number of leaf nodes (i.e., the number of partitions) and the prediction value at each leaf node in regularization: $\Omega(\Psi_\ell,\gamma_\ell) = \tau|\Psi_\ell| + \|\theta(\Psi_\ell,\gamma_\ell)\|$, where $|\Psi_\ell|$ is the number of leaf nodes from partition Ψ_ℓ, $\theta(\Psi_\ell,\gamma_\ell)$ is the vector of the leaf node prediction values induced by Ψ_ℓ and γ_ℓ, and τ is a scaling coefficient. The shrinkage parameters α_ℓ's also serve as an additional layer of regularization and are tuned separately

from the regularization term $\Omega(\cdot)$. We formalize the PAB method in Algorithm 1.

Algorithm 1 (Pooling and Boosting Algorithm (PAB))

1: **Input**: SKU sales observations y_{ij}, feature vectors $\mathbf{x}_{ij}$, and proxy split vector $\tilde{\boldsymbol{\omega}}$; boosting parameters t_1, t.
2: Construct proxy demand data $\tilde{y}_{ij}$:
3: Pool product sales to obtain y_c according to the category (subcategory) membership;
4: Obtain $\tilde{y}_{ij}$ by splitting y_{cj} with respect to $\tilde{\omega}_i$;
5: Train the first gradient boosting trees $\sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy})$ on the proxy demand data;
6: Train the second (regularized) gradient boosting trees on top of $\sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy})$ using the original demand data: $\sum_{\ell\in[t]/[t_1]}\alpha_\ell T(\mathbf{x};\hat{\Psi}_\ell,\hat{\gamma}_\ell)$ as the solution to Problem (9);
7: **Output**: combined gradient boosting trees for future demand prediction $f_{PAB}(\mathbf{x}) = \sum_{\ell\in[t_1]}\alpha_\ell T(\mathbf{x};\hat{\Psi}_\ell^{proxy},\hat{\gamma}_\ell^{proxy}) + \sum_{\ell\in[t]/[t_1]}\alpha_\ell T(\mathbf{x};\hat{\Psi}_\ell,\hat{\gamma}_\ell)$.

Although PAB connects boosting trees built on two different data sets, it still enjoys the generalization error bound of regularized boosting trees under certain conditions. Specifically, let $R(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[L(f(\mathbf{x}),y)]$ be the true risk function defined on the population distribution $\mathcal{D}$ over $\mathcal{X}\times\mathcal{Y}$, where $\mathcal{X}$ is the input space and $\mathcal{Y}$ is a measurable subset of $\mathbb{R}$. Correspondingly, let $\hat{R}_{\mathcal{S}}(f)$ denote the empirical risk function defined on a sample $\mathcal{S}$ drawn from $\mathcal{D}$: $\mathcal{S} = \{(\mathbf{x}_1,y_1),\ldots,(\mathbf{x}_n,y_n)\}$. Suppose that each boosting tree has at most h leaf nodes and the leaf prediction vector has bounded ℓ_1 norm of η from regularization. More specifically, each added boosting tree belongs to a set of regularized boosting trees $\mathcal{F}_{h_q,\eta_q}$ that has at most h_q leaf nodes with the ℓ_1 norm of the prediction vector bounded by η_q ($h_q \le h$ and $\eta_q \le \eta$) for some $q\in[Q]$. So $\{\mathcal{F}_{h_q,\eta_q}\}_{q=1}^{Q}$ represent families of regularized regression trees with varying complexities that a base model can be mapped to. The following result gives the out-of-sample performance guarantee of PAB in terms of the empirical risk for the squared error function, that is, $L(y,f(\mathbf{x})) = (y-f(\mathbf{x}))^2$ (Cortes et al. 2019).

Proposition 1. *Assume* $|y-f(\mathbf{x})| \le M$ *for all* $(\mathbf{x},y)\in\mathcal{X}\times\mathcal{Y}$ *and* $f\in\mathcal{F}$. *Then for any* $p>0$, *with probability at least* $(1-p)$ *over the sampled training data* $\mathcal{S}$ *of size* n, *it holds that for* $f_{PAB}\in conv(\cup_{q\in[Q]}\mathcal{F}_{h_q,\eta_q})$,

$$R(f_{PAB}) \le \hat{R}(f_{PAB}) + 4M\eta\sqrt{\frac{(4h+2)\log_2(d+2)\log(n+1)}{n}} + O\left(\sqrt{\frac{\log(2/p)}{n}}\right).$$

Proposition 1 relies on bounding the empirical Rademacher complexity of boosting trees of limited sizes and prediction weights. It shows how the generalization error depends on the sample size and regularization terms. By design, PAB is computationally efficient because it can be directly implemented within scalable boosting frameworks such as XGBoost. Numerical results comparing the generalization bound and empirical error performance on synthetic data are presented in Online Appendix EC.2.

3. Prediction Model Evaluation Results

In this section, we report the out-of-sample performance evaluation results of PAB based on an internal study (experiment) with JD.com. We first describe the data set and the preprocessing steps. We then introduce the implementation of PAB, including the cross-validation procedure and the tested benchmarks. Finally, we discuss the evaluation results and prescribe managerial insights.

3.1. Data Preparation

JD.com offered two sets of SKUs from its online platform for the internal experiment. The first one contains a subcategory of milk products, including weekly sales of 27 SKUs from January 2020 to December 2021. The second set includes a subcategory of 39 power strips from the same period. These two subcategories are representative of grocery and electronic items, respectively. Grocery and electronic items are among the most popular SKUs sold on JD.com. Moreover, milk and power strips have relatively long life cycles, which eases the need for detailed life cycle curve modeling.[5] Table 1 presents the main summary statistics about the two data sets. In total, the experiment observes a total sales volume of more than 85 million.

In addition to the weekly sales observations, we have access to two types of product features: (i) temporal features such as month, week, and past sales statistics; these features were engineered by JD.com and used in its forecasting algorithms; (ii) product attributes that include brand, price, and promotion information. Specifically, the promotion label takes a binary value that equals one if a discount of more than 10% is provided. The list of features is summarized in Table 2. In total, we have 18 features, including 5 categorical variables and 13 numerical variables.

3.2. Implementation of PAB and Benchmarks

Following JD.com's guidelines, we adopt a sliding-window evaluation procedure to measure the out-of-sample prediction performance.

3.2.1. Evaluation Procedure. We use $n = 52$ past weekly sales observations as training data and generate demand prediction for the following four weeks (validation data). The prediction performance is recorded on the validation

Table 1. Summary Statistics About the Two Data Sets

Category	Number of SKUs	Number of observations ($= m \times n$)	Average weekly sales (units)	Average actual price (RMB)
Milk	27	2,565	28,939	49.95
Power strip	39	3,705	3,119	39.43

data, which are used to calibrate the tuning parameters of each model. Then the model with the tuned parameters is applied to the test set of four weeks that follow the validation set. We repeat this process until we reach the end of the prediction time window (i.e., the end of 2021). More specifically, in round k of the evaluation, the training data contain the sales observations from week k to $k+51$; the validation data include sales observations between week $k+52$ and $k+55$, and the test period is from week $k+56$ to $k+59$. We report the forecasting performance on the test set.

3.2.2. PAB and Benchmarks. We implement PAB using the XGBoost framework in Python. XGBoost has a number of tuning parameters, and we focus on the key parameters that control overfitting in this study (the omitted parameters are set to the default values). Specifically, we fix the number of total boosting iterations (t) to 200 for PAB and vary the number of boosting iterations on the proxy data (t_1) in $\{10, 20, \ldots, 190\}$. Accordingly, the number of boosting iterations on the gold data is $t - t_1$. The other tuning (regularization) parameters include maximum depth of trees (max_depth), learning rate (shrinkage parameter), and minimum sum of instance weights required in a child node (min_child_weight). We perform a grid search to calibrate these parameters. Table 3 summarizes the parameter setting of PAB. To generate the proxy observations from training data, we compute the moving-average historical market share of each SKU to construct the split vector. Specifically, we calibrate the split vector in period j by the recent four-week empirical market share: $\tilde{\omega}_i(j) = \sum_{j'=j-4}^{j-1} y_{ij'} / \sum_{j'=j-4}^{j-1} y_{cj'}$, where $y_{cj'}$ represents the category sales in period j'. We also test alternative estimation methods for the proxy split vector, of which the results are presented in Online Appendix EC.6. The computed split vector is then multiplied with the category sales observations to obtain proxy SKU sales observations.

Table 2. The Description of Features

Feature	Description
Brand	Brand of SKU
Initial price	Initial price when first launched on JD.com
Actual price	Current actual price (after accounting for promotion)
Promotion	Whether there was a promotion
SKU_mean	Average sales in all past weeks
SKU_sd	Standard deviation of sales in all past weeks
Month	Month of sales
Season	Season of sales
Week	Week of sales
Lag_k	Sales before k weeks: $k \in \{1, 4, 8\}$
MA_k	Average sales in the last k weeks: $k \in \{4, 8\}$
Sd_k	Standard deviation of sales in the last k weeks: $k \in \{4, 8\}$
Change_k	Rate of change: the difference between sales in the most recent week and sales before k weeks ($k \in \{2, 4\}$)

Table 3. Tuning Parameters of PAB

Parameter	Range of values
t_1	[10, 190]
max_depth	[3, 10]
Learning rate	[0.01, 0.2]
min_child_weight	[1, 6]

We compare PAB to the following benchmark methods:

1. Gold method. This method fits gradient boosting trees to the original SKU sales observations (gold data). The gold method is adapted from JD.com's forecasting algorithm and serves as its benchmark.

2. Proxy method. This method fits gradient boosting trees to the proxy sales data only.

3. Top-down method. This method first generates category demand forecast using gradient boosting trees (with category sales features) and then disaggregates it to SKUs according to the computed split vector.

4. MinT. This method is an advanced hierarchical forecasting method introduced in Wickramasuriya et al. (2019). We consider two ways to generate base forecasts for MinT: (i) the state space model for exponential smoothing (ETS), which is a time series method commonly used for MinT (Wickramasuriya et al. 2019); (ii) gradient boosting trees.

5. Lasso. Lasso regression applies ℓ_1-norm regularization to improve out-of-sample performance.

6. HB method. This method fits a linear regression model to each SKU, in which the regression coefficients are drawn from a common multivariate normal prior. We implement HB using the R package **bayesm** with default prior parameters.

7. Temporal Fusion Transformer (TFT). This method is a state-of-the-art time series prediction model based on neural networks, as introduced in Lim et al. (2021). We implement TFT using the Python package **NeuralForecast**.

8. Least Absolute Deviation (LAD) regression. LAD regression minimizes the sum of absolute errors to provide robust estimates. We adopt recursive feature elimination (based on the Python package **sklearn.feature_selection**) to select the most important features before applying LAD.

Both the top-down and MinT methods are widely used in hierarchical forecasting. They aim to use information from different levels to achieve coherent forecasts, which does not necessarily improve SKU forecasts at the bottom level. Note that the other hierarchical

forecasting alternative, the bottom-up method, is not applicable for improving bottom-level forecasting tasks. We perform the same tuning process as PAB for each benchmark with hyperparameters.

3.2.3. Error Measures. We adopt four scale independent error measures to evaluate the prediction performance: mean absolute scaled error (MASE), mean absolute percentage error (MAPE), symmetric MAPE (sMAPE), and weighted MAPE (wMAPE). These error measures indicate relative prediction performance of the method (Hu et al. 2019, Makridakis et al. 2022a). Specifically, MAPE is computed by $\frac{1}{n}\sum_{j=1}^{n}|(\hat{y}_j - y_j)/y_j|$ given actual values y_j and prediction $\hat{y}_j$. MAPE may become unreliable when the actual value is close to zero and puts a heavier penalty on negative errors. To alleviate this issue, sMAPE replaces the denominator of MAPE by $(|\hat{y}_j| + |y_j|)/2$ and has a bounded value. wMAPE weights the errors by the actual value so it rewards better forecasts of higher-volume SKUs, which is given by $\sum_{j=1}^{n}|\hat{y}_j - y_j| / \sum_{j=1}^{n}|y_j|$.

MAPE and its variants are skewed toward SKUs with low sales. MASE overcomes this limitation by scaling the absolute error by the errors from a naive forecast that simply uses the most recent demand observation as the prediction of the future period (Hyndman and Koehler 2006). Specifically, it is computed by

$$\frac{1}{n}\sum_{j=1}^{n}\frac{|\hat{y}_j - y_j|}{(1/(n-1))\sum_{j'=2}^{n}|y_{j'} - y_{j'-1}|} = \frac{\sum_{j=1}^{n}|\hat{y}_j - y_j|}{(n/(n-1))\sum_{j'=2}^{n}|y_{j'} - y_{j'-1}|}.$$

3.3. Evaluation Results and Insights

We report the experiment results and draw insights into the performance improvement of PAB.

3.3.1. Evaluation Results. Table 4 presents the out-of-sample evaluation results based on the data sets of JD.com. We treat the gold method as the baseline and report the average relative error reduction of each method against the gold method (a positive value indicates improved performance over JD.com's forecasts). We observe that PAB outperforms the benchmarks by a large margin across different error measures and the two categories. Specifically, whereas the other methods fail to improve on the gold method consistently, PAB reduces the prediction error by up to 14% over the gold method. Notably, using the proxy data alone does not result in significant improvement, with errors jumping by more than 10% in most cases. This underscores the value of transfer learning in combining proxy and gold data effectively.

In addition, the two hierarchical forecasting methods do not significantly improve the gold result and often degrade the forecasting performance, which stresses that optimizing for coherency may not yield better forecasts across different levels. One should be cautioned against applying hierarchical forecasting directly if the SKU forecasts are the main focus. Linear models such as Lasso and HB are dominated by the tree-based benchmarks, with HB performing better than Lasso. This is due to the potentially nonlinear relationship between SKU demand and features. The neural network-based method, TFT, fails to beat the gold method and PAB, which echoes the empirical evidence supporting the superior performance of

Table 4. Out-of-Sample Forecasting Performance Comparison

		Average relative error reduction over the gold method			
Category	Method	MASE	MAPE	sMAPE	wMAPE
Milk	Proxy	−10.72%	−11.82%	−8.74%	−9.45%
	PAB	**10.53%**	**12.02%**	**10.36%**	**8.71%**
	Top-down	−31.78%	−33.18%	−22.43%	−27.24%
	MinT (ETS)	−70.12%	−84.78%	−49.24%	−66.45%
	MinT (boosting trees)	−11.15%	−13.87%	−9.75%	−11.32%
	Lasso	−68.26%	−80.84%	−59.38%	−54.40%
	HB	−57.82%	−70.79%	−40.59%	−38.61%
	TFT	−37.13%	−40.00%	−29.64%	−44.51%
	LAD	−17.55%	−21.51%	−14.13%	−19.84%
Power strip	Proxy	−13.36%	−12.49%	−13.20%	−16.48%
	PAB	**12.33%**	**11.49%**	**12.82%**	**14.12%**
	Top-down	−11.64%	−11.26%	−13.18%	−14.82%
	MinT (ETS)	−22.96%	−24.66%	−19.83%	−22.06%
	MinT (boosting trees)	−0.57%	−1.36%	−0.55%	−3.87%
	Lasso	−68.47%	−74.24%	−49.66%	−37.01%
	HB	−13.65%	−16.22%	−14.89%	−28.11%
	TFT	−8.34%	−6.97%	−9.44%	−17.90%
	LAD	−11.87%	−20.01%	−5.74%	−4.80%

Note. The bold values are the best predictive performance.

boosting trees in retail demand forecasting applications (Makridakis et al. 2022a).

3.3.2. Feature Importance Scores. We investigate the tree structures generated from the gold method, proxy method, and PAB. Specifically, we compute the feature importance scores from the trained gradient boosting trees: the feature importance score measures the contribution of each feature variable to tree splits. The feature with a higher score is more frequently used to split the data. Figure 2 presents the relative importance scores of the top 10 features in two different prediction rounds for the milk SKUs (corresponding to two prediction models trained for different weeks). It shows that temporal (past sales statistics) and price features contribute the most to the training of boosting trees. Because the temporal pattern may be well calibrated with the proxy data, the importance of temporal features suggests the value of proxy demand to PAB. Notably, we observe that the importance scores of the gold and proxy methods are mostly similar, with a few notable differences (e.g., feature Lag_1 in prediction round 1 and Change_2 in round 47). Accordingly, the feature importance score ranking of the gold method differs only slightly from the proxy method. Nevertheless, the feature importance score ranking of PAB has subtle differences from that of the gold and proxy methods. For example, the initial price feature appears as a top feature only for PAB in round 1. Additionally, in round 47, the week feature is included in PAB's ranking but excluded from the gold and proxy methods. These differences partially explain the contrasting forecasting performance of PAB versus the gold and proxy methods. Similar observations hold for gradient boosting trees trained on the power strip data, of which the feature importance score rankings are presented in Figure EC.6 of Online Appendix EC.4.

3.4. Additional Study with Category-Brand Data

In this section, we explore the use of PAB in a different hierarchy: category-brand prediction. For retailers like JD.com, forecasting brand demand is valuable for contract negotiation and promotion planning with suppliers. To this end, we aggregate the milk sales data by brand and obtain the corresponding weekly sales observations. In total, there are eight different brands. The evaluation procedure follows Section 3.2 except that we drop the SKU-level features and add temporal features related to brand sales. The evaluation result is summarized in Table 5.

We observe that PAB outperforms other methods consistently, yielding more than 20% error reductions over the gold method. The improvement is greater than what we observe from the category-SKU data, which can be attributed to the brand sales pattern being less volatile than SKU sales. Note that the proxy method still fails to deliver performance improvement over the gold method. This implies that the value of proxy data can be effectively exploited only when combined with the gold data.

3.5. Extensions of PAB

In addition to pooling across products, PAB can be extended to leverage the benefit of pooling across time. In particular, the demand pattern over days of the week can be similar for different weeks, so weekly pooling is a viable option for generating the proxy data. To investigate its benefit, we test different pooling strategies of PAB in forecasting daily sales for the subcategory of power strip SKUs. Specifically, we consider three pooling strategies: (i) product pooling (base model)—the category daily sales data are split to form the proxy data; (ii) weekly pooling—the proxy data are generated by splitting the weekly sales data of SKUs according to a split vector over days of the week; (iii) product and weekly pooling—the proxy data are generated by splitting the aggregate category weekly sales data according to a split vector over products and days. The split vector is constructed using the same method as described in Section 3.2.

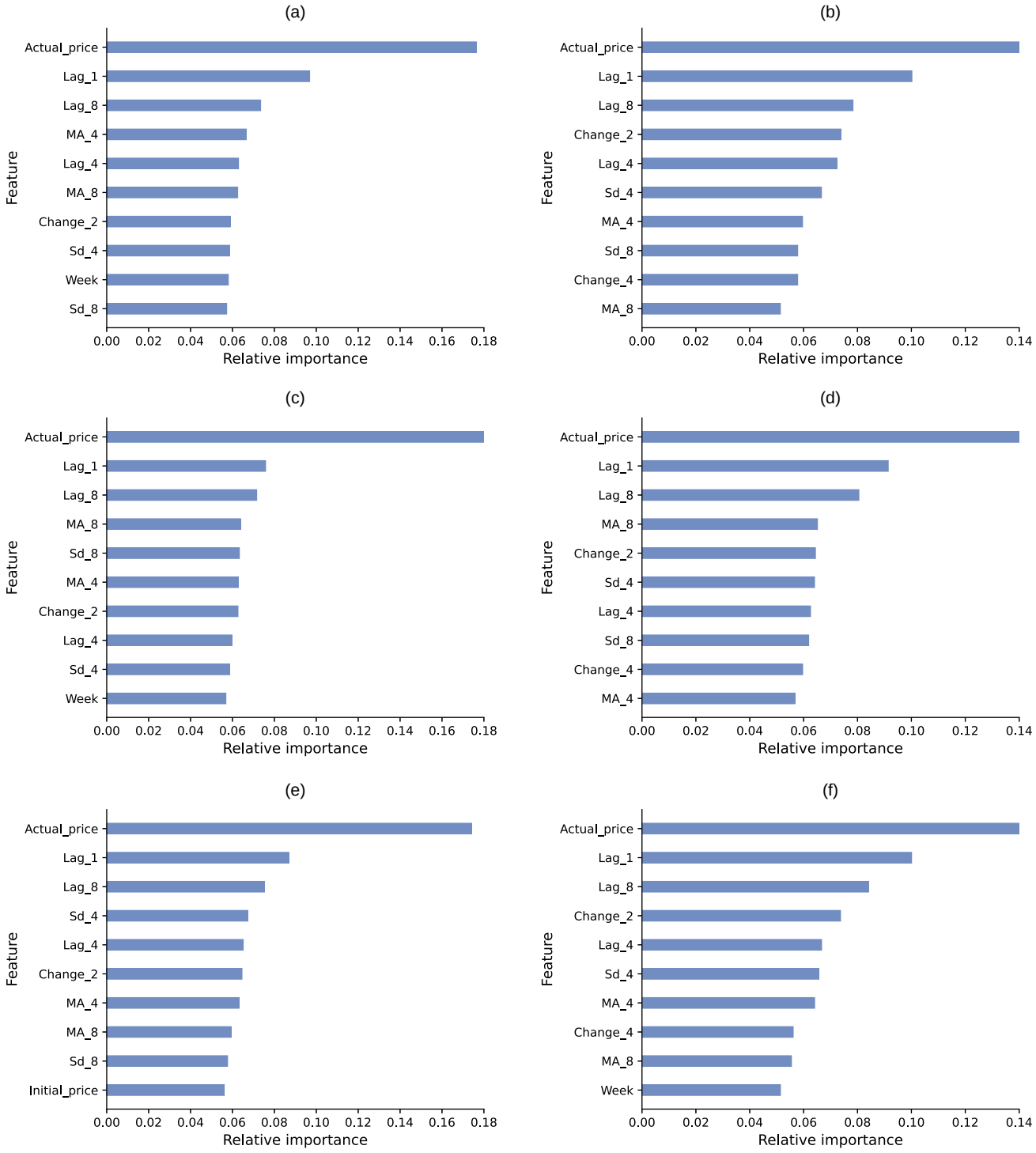
Table 6 summarizes the evaluation results of different PAB methods and benchmarks on daily sales forecasting. We observe that PAB outperforms the benchmarks regardless of the pooling strategy. Among the three strategies, weekly pooling performs the best based on MASE, MAPE, and sMAPE, while product pooling achieves the best wMAPE. Pooling across both products and time performs slightly worse than pooling across time alone, possibly because the estimated split vector is noisier.

The idea of PAB can also be applied to forecast methods other than tree models. In Online Appendix EC.3, we test fine-tuning the TFT model using proxy data, following a similar rationale as for PAB. We observe this strategy improves the performance of TFT consistently across the tested data sets.

3.6. Operational Implications

The improvement in forecasting accuracy generates direct financial benefits for retailers. With more accurate SKU demand forecasts, JD.com can better negotiate contracts, plan marketing events, and make inventory and fulfillment decisions, saving warehouse space and transportation costs. Through the collaboration, we conducted financial data analysis to estimate the operating cost savings from the observed forecasting improvement. Specifically, our conservative estimate is that the forecasting error reduction from PAB can lead to an average annual cost reduction of 0.01–0.29 RMB per unit of sold SKU (compared with the gold method). Envisioning a large-scale implementation at JD.com, this cost reduction implies millions of dollars of savings for JD.com's self-managed SKUs per year.

Figure 2. (Color online) Relative Feature Importance Scores of Gradient Boosting Trees on the Milk Data Set



Notes. (a) Gold method (prediction round 1). (b) Gold method (prediction round 47). (c) Proxy method (prediction round 1). (d) Proxy method (prediction round 47). (e) PAB (prediction round 1). (f) PAB (prediction round 47).

4. Application to the Walmart Retail Data

To further validate the performance and generalizability of PAB, we conduct additional experiments using the Walmart retail data that were made publicly available through the M5 Accuracy competition in 2020 (Mcompetitions 2020b, Makridakis et al. 2022b). Specifically, the sales data of 264 SKUs in the "Food_3" category of a Walmart store ("CA_3"), spanning over 2,000 days, are used for our experimentation. We treat weekly SKU sales as the lower-level prediction task, and the aggregate level includes all

Table 5. Out-of-Sample Forecasting Performance Comparison on the Category-Brand Data

	Average relative error reduction over the gold method			
Method	MASE	MAPE	sMAPE	wMAPE
Proxy	−8.48%	−9.77%	−7.73%	−9.40%
PAB	**22.36%**	**24.50%**	**23.20%**	**20.90%**
Top-down	−35.40%	−39.46%	−22.53%	−19.14%
MinT (ETS)	−29.54%	−38.79%	−19.34%	−16.68%
MinT (boosting trees)	−6.27%	−8.70%	−3.28%	−1.05%
Lasso	−94.53%	−105.25%	−67.62%	−119.51%
HB	−19.76%	−27.33%	−16.50%	−12.03%
TFT	−17.77%	−22.53%	−44.05%	−19.48%
LAD	−21.69%	−25.35%	−19.19%	−27.49%

Note. The bold values are the best predictive performance.

the SKUs. The training, validation, and testing sets are divided according to the rules set by the competition organizer. The feature variables are extracted from the method of the winner of the competition (Mcompetitions 2020a). In this experiment, we evaluate PAB and the benchmark methods regarding their prediction performance compared with the winner's method, which is based on LightGBM. Table 7 presents the out-of-sample evaluation results on the Walmart retail data. (Because there are a large number of zero values in the data set, MASE and MAPE are no longer appropriate; instead, we report only sMAPE and wMAPE as the error measures.)

Consistent with our findings from the internal study with JD.com, PAB exhibits dominating performance among the tested approaches and improves over the method of the winner of the competition significantly (reducing errors by 5%–9%). Interestingly, the gold method yields performance comparable to the winner's method due to the fact that both of them are based on gradient boosting trees. We also test PAB on the open Australia tourism data that are frequently used for hierarchical forecasting applications, and similar observations hold (the detailed result is presented in Online Appendix EC.5). These results bode well for other related prediction tasks.

5. Conclusion

In this paper, we provide a novel framework for forecasting SKU demand by integrating data from different aggregation levels through transfer learning. The proposed approach, PAB, connects the joint estimator (developed for linear models) and decision trees with the use of regularized gradient boosting. Based on the error bound analysis, PAB is proved to be most effective when the benefit of pooling is large and the bias is sparse. We conduct rigorous experiments with JD.com to evaluate the practical forecasting performance of PAB. Our evaluation result illustrates that PAB outperforms the state-of-the-art methods consistently. Particularly, PAB leads to forecasting error reductions of more than 9% over the benchmark model of JD.com across all metrics. Based on internal financial data analysis, we estimate that the improved forecasting performance of PAB can reduce the annual operating cost by 0.01–0.29 RMB per unit on average. This improvement can translate to millions of dollars of savings per year for JD.com under a large-scale implementation. As additional support, we show that PAB delivers superior prediction performance on a retail data set from Walmart, outperforming the method of the winner of the M5 Accuracy competition by 5%–9%. We also demonstrate the generalizability of PAB to different pooling mechanisms and

Table 6. Out-of-Sample Forecasting Performance Comparison on the Daily Power Strip Data

	Average relative error reduction over the gold method			
Method	MASE	MAPE	sMAPE	wMAPE
Proxy (weekly pooling)	−0.97%	0.36%	−0.82%	−3.02%
PAB (product pooling)	2.81%	3.22%	2.58%	**2.29%**
PAB (weekly pooling)	**3.35%**	**4.05%**	**2.96%**	2.26%
PAB (weekly product pooling)	3.04%	3.74%	2.56%	2.03%
Top-down	−17.20%	−22.50%	−10.80%	−6.90%
MinT (ETS)	−37.01%	−65.60%	−36.03%	−41.40%
MinT (boosting trees)	−3.94%	−7.29%	−2.39%	0.65%
Lasso	−23.69%	−36.61%	−19.86%	−1.87%
HB	−2.53%	−8.28%	−12.00%	−2.34%
TFT	−19.31%	−29.13%	−23.01%	−20.16%
LAD	−23.82%	−23.82%	−26.58%	−43.42%

Note. The bold values are the best predictive performance.

Table 7. Out-of-Sample Forecasting Performance Comparison on the Walmart Retail Data

	Average relative error reduction over the winner's method	
Method	sMAPE	wMAPE
Gold	−1.58%	1.12%
Proxy	−10.46%	−10.67%
PAB	**5.32%**	**9.51%**
Top-down	−18.01%	−21.69%
MinT (ETS)	−71.46%	−68.08%
MinT (boosting trees)	−11.75%	−8.83%
Lasso	−31.04%	−23.55%
HB	−61.26%	−56.83%
TFT	−46.53%	−46.64%
LAD	−20.65%	−20.30%

Note. The bold values are the best predictive performance.

prediction methods, for example, pooling across time and neural network-based models.

PAB can be applied to other forecasting problems in hierarchies when the bottom-level forecasting accuracy is most concerning, including supply chain risk prediction, patient flow management, and energy consumption forecasting. One can extend our framework to a hierarchy of more than two levels with a possibly different split policy. The split information can be formed by a combination of market research and expert judgment, which can be further tailored to different applications. It would also be interesting to explore clustering techniques to identify the best aggregation strategy (across different categories) that can effectively exploit the substitution and pooling effect among different SKUs. An end-to-end approach may be applicable to learn the best aggregation level for PAB from data. Lastly, it is valuable to study new regularization techniques for gradient boosting trees in a transfer learning problem. For example, one can define tree complexity using different geometric and combinatorial measures to encourage sparsity.

Acknowledgments

The authors are grateful to Ningyuan Chen and Xiaojia Guo for the early discussion of this work, and to Jérémie Gallien, the associate editor, and three referees for valuable comments that greatly improved the quality of the manuscript.

Endnotes

[1] Demand prediction for new products (when no sales information is available) is more challenging and is often addressed by exploiting the similarity between new products and existing ones.

[2] We differentiate between data aggregation and pooling here. Data aggregation refers to training a single model across SKUs, whereas pooling indicates summing over demands of SKUs. Hence, our notion of pooling shares a rationale similar to inventory risk pooling.

[3] This standardization is driven by data aggregation, in which a single model is fitted to all SKUs in the category to reduce variance.

[4] When the model coefficient is product dependent, one can apply the joint estimator to the demand data of each product separately.

[5] Demand prediction for products of short life cycles may require more detailed calibration of life cycle curves; see Hu et al. (2019).

References

Abolghasemi M, Tarr G, Bergmeir C (2024) Machine learning applications in hierarchical time series forecasting: Investigating the impact of promotions. *Internat. J. Forecasting* 40(2):597–615.

Adam H, He P, Zheng F (2023) Machine learning for demand estimation in long tail markets. *Management Sci.*, ePub ahead of print October 10, https://doi.org/10.1287/mnsc.2023.4893.

Ali ÖG, Sayın S, Van Woensel T, Fransoo J (2009) SKU demand forecasting in the presence of promotions. *Expert Systems Appl.* 36(10):12340–12348.

Alley M, Biggs M, Hariss R, Herrmann C, Li ML, Perakis G (2023) Pricing for heterogeneous products: Analytics for ticket reselling. *Manufacturing Service Oper. Management* 25(2):409–426.

Athanasopoulos G, Hyndman RJ, Kourentzes N, Petropoulos F (2017) Forecasting with temporal hierarchies. *Eur. J. Oper. Res.* 262(1):60–74.

Baardman L, Levin I, Perakis G, Singhvi D (2017) Leveraging comparables for new product sales forecasting. Preprint, submitted December 11, http://dx.doi.org/10.2139/ssrn.3086237.

Ban G-Y, Gallien J, Mersereau AJ (2019) Dynamic procurement of new products with covariate information: The residual tree method. *Manufacturing Service Oper. Management* 21(4):798–815.

Bastani H (2021) Predicting with proxies: Transfer learning in high dimension. *Management Sci.* 67(5):2964–2984.

Bastani H, Simchi-Levi D, Zhu R (2022) Meta dynamic pricing: Transfer learning across experiments. *Management Sci.* 68(3):1865–1881.

Boone T, Ganeshan R, Hicks RL, Sanders NR (2018) Can Google Trends improve your sales forecast? *Production Oper. Management* 27(10):1770–1774.

Caro F, Gallien J (2010) Inventory management of a fast-fashion retail network. *Oper. Res.* 58(2):257–273.

Chen T, Guestrin C (2016) Xgboost: A scalable tree boosting system. *Proc. 22nd ACM SIGKDD Internat. Conf. Knowledge Discovery Data Mining* (Association for Computing Machinery, New York), 785–794.

Chen Y-C, Mišić VV (2022) Decision forest: A nonparametric approach to modeling irrational choice. *Management Sci.* 68(10):7090–7111.

Cohen MC, Zhang R, Jiao K (2022a) Data aggregation and demand prediction. *Oper. Res.* 70(5):2597–2618.

Cohen MC, Gras P-E, Pentecoste A, Zhang R (2022b) *Demand Prediction in Retail: A Practical Guide to Leverage Data and Predictive Analytics* (Springer Nature, New York).

Cooper LG (1993) Market-share models. *Handbooks in Operations Research and Management Science*, vol. 5 (Elsevier, Amsterdam), 259–314.

Cortes C, Mohri M, Storcheus D (2019) Regularized gradient boosting. *NIPS '19 Proc. 33rd Internat. Conf. Neural Inform. Processing Systems* (Curran Associates Inc., Red Hook, NY), 5449–5458.

Cui R, Gallino S, Moreno A, Zhang DJ (2018) The operational value of social media information. *Production Oper. Management* 27(10): 1749–1769.

Dai W, Yang Q, Xue G, Yong Y (2007) Boosting for transfer learning. Ghahramani Z, ed. *Proc. 24th Internat. Conf. Machine Learn.* (Association for Computing Machinery, New York), 193–200.

Eckert F, Hyndman RJ, Panagiotelis A (2021) Forecasting Swiss exports using Bayesian forecast reconciliation. *Eur. J. Oper. Res.* 291(2):693–710.

Evgeniou T, Pontil M, Toubia O (2007) A convex optimization approach to modeling consumer heterogeneity in conjoint estimation. *Marketing Sci.* 26(6):805–818.

Ferreira KJ, Lee BHA, Simchi-Levi D (2016) Analytics for an online retailer: Demand forecasting and price optimization. *Manufacturing Service Oper. Management* 18(1):69–88.

Fildes R, Ma S, Kolassa S (2022) Retail forecasting: Research and practice. *Internat. J. Forecasting* 38(4):1283–1318.

Gross CW, Sohl JE (2010) Disaggregation methods to expedite product line forecasting. *J. Forecasting* 9(3):233–254.

Guo X, Lichtendahl KC, Grushka-Cockayne Y (2022) Bayesian ensembles of exponentially smoothed life-cycle forecasts. Working paper, Robert H. Smith School of Business, University of Maryland, College Park, MD.

Gupta V, Kallus N (2022) Data pooling in stochastic optimization. *Management Sci.* 68(3):1595–1615.

Hastie T, Tibshirani R, Friedman J (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York).

Hu K, Acimovic J, Erize F, Thomas DJ, Van Mieghem JA (2019) Forecasting new product life cycle curves: Practical approach and empirical analysis: Finalist–2017 M&SOM practice-based research competition. *Manufacturing Service Oper. Management* 21(1):66–85.

Huang T, Fildes R, Soopramanien D (2014) The value of competitive information in forecasting FMCG retail product sales and the variable selection problem. *Eur. J. Oper. Res.* 237(2):738–748.

Hyndman RJ, Koehler AB (2006) Another look at measures of forecast accuracy. *Internat. J. Forecasting* 22(4):679–688.

Hyndman RJ, Ahmed RA, Athanasopoulos G, Shang HL (2011) Optimal combination forecasts for hierarchical time series. *Comput. Statist. Data Anal.* 55(9):2579–2589.

JD.com (2022) JD.com announces fourth quarter and full year 2021 results. Accessed September 2, 2022, https://ir.jd.com/news-releases/news-release-details/jdcom-announces-fourth-quarter-and-full-year-2021-results.

Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu T-Y (2017) LightGBM: A highly efficient gradient boosting decision tree. *Proc. 33rd Internat. Conf. Neural Inform. Processing Systems (NIPS 2017)* (Curran Associates Inc., Red Hook, NY), 3149–3157.

Lei D, Hu H, Geng D, Zhang J, Qi Y, Liu S, Shen Z-JM (2023) New product life cycle curve modeling and forecasting with product attributes and promotion: A Bayesian functional approach. *Production Oper. Management* 32(2):655–673.

Lim B, Arık SÖ, Loeff N, Pfister T (2021) Temporal fusion transformers for interpretable multi-horizon time series forecasting. *Internat. J. Forecasting* 37(4):1748–1764.

Ma S, Fildes R, Huang T (2016) Demand forecasting with high dimensional data: The case of SKU retail sales forecasting with intra-and inter-category promotional information. *Eur. J. Oper. Res.* 249(1):245–257.

Makridakis S, Spiliotis E, Assimakopoulos V (2022a) M5 accuracy competition: Results, findings, and conclusions. *Internat. J. Forecasting* 38(4):1346–1364.

Makridakis S, Spiliotis E, Assimakopoulos V (2022b) The m5 competition: Background, organization, and implementation. *Internat. J. Forecasting* 38(4):1325–1336.

Mcompetitions (2020a) M5-methods: Code of winning methods. https://github.com/Mcompetitions/M5-methods/tree/master/Code%20of%20Winning%20Methods.

Mcompetitions (2020b) M5-methods: Data, benchmarks, and methods submitted to the m5 forecasting competition. https://github.com/Mcompetitions/M5-methods.

Mircetic D, Rostami-Tabar B, Nikolicic S, Maslaric M (2022) Forecasting hierarchical time series in supply chains: An empirical investigation. *Internat. J. Production Res.* 60(8):2514–2533.

Montgomery AL (1997) Creating micro-marketing pricing strategies using supermarket scanner data. *Marketing Sci.* 16(4):315–337.

Nevo A (2000) A practitioner's guide to estimation of random-coefficients logit models of demand. *J. Econom. Management Strategy* 9(4):513–548.

Nystrup P, Lindström E, Pinson P, Madsen H (2020) Temporal hierarchies with autocorrelation for load forecasting. *Eur. J. Oper. Res.* 280(3):876–888.

Pan SJ, Yang Q (2010) A survey on transfer learning. *IEEE Trans. Knowledge Data Engrg.* 10(22):1345–1359.

Pennings CLP, Van Dalen J (2017) Integrated hierarchical forecasting. *Eur. J. Oper. Res.* 263(2):412–418.

Rossi PE, Allenby GM (2003) Bayesian statistics and marketing. *Marketing Sci.* 22(3):304–328.

Rossi PE, Allenby GM, McCulloch R (2006) *Bayesian Statistics and Marketing* (Wiley, Hoboken, NJ).

Segev N, Harel M, Mannor S, Crammer K, El-Yaniv R (2016) Learn on source, refine on target: A model transfer learning framework with random forests. *IEEE Trans. Pattern Anal. Machine Intelligence* 39(9):1811–1824.

Shlifer E, Wolff RW (1979) Aggregation and proration in forecasting. *Management Sci.* 25(6):594–603.

Steinker S, Hoberg K, Thonemann UW (2017) The value of weather information for e-commerce operations. *Production Oper. Management* 26(10):1854–1874.

Syntetos AA, Babai Z, Boylan JE, Kolassa S, Nikolopoulos K (2016) Supply chain forecasting: Theory, practice, their gap and the future. *Eur. J. Oper. Res.* 252(1):1–26.

Taieb SB, Taylor JW, Hyndman RJ (2021) Hierarchical probabilistic forecasting of electricity demand with smart meter data. *J. Amer. Statist. Assoc.* 116(533):27–43.

Tan C, Sun F, Kong T, Zhang W, Yang C, Liu C (2018) A survey on deep transfer learning. Kůrková V, Manolopoulos Y, Hammer B, Iliadis L, Maglogiannis I, eds. *27th Internat. Conf. Artificial Neural Networks-ICANN 2018* (Springer, New York), 270–279.

Tibshirani R, Wasserman L (2017) Sparsity, the lasso, and friends. Lecture notes from "Statistical Machine Learning," Carnegie Mellon University, Pittsburgh, Spring.

van Erven T, Cugliari J (2015) Game-theoretically optimal reconciliation of contemporaneous hierarchical time series forecasts. Antoniadis A, Poggi J-M, Brossat X, eds. *Modeling and Stochastic Learning for Forecasting in High Dimensions* (Springer, Cham, Switzerland), 297–317.

Wang J, Holler A, Wang M, Mui M (2019) Productionizing distributed xgboost to train deep tree models with large data sets at Uber. Accessed September 2, 2022, https://www.uber.com/en-CA/blog/productionizing-distributed-xgboost/.

Wellens AP, Boute RN, Udenio M (2024) Simplifying tree-based methods for retail sales forecasting with explanatory variables. *Eur. J. Oper. Res.* 314(2):523–539.

Wellens AP, Udenio M, Boute RN (2022) Transfer learning for hierarchical forecasting: Reducing computational efforts of m5 winning methods. *Internat. J. Forecasting* 38(4):1482–1491.

Wickramasuriya SL, Athanasopoulos G, Hyndman RJ (2019) Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization. *J. Amer. Statist. Assoc.* 114(526):804–819.

Widiarta H, Viswanathan S, Piplani R (2009) Forecasting aggregate demand: An analytical evaluation of top-down vs. bottom-up forecasting in a production planning framework. *Internat. J. Production Econom.* 118(1):87–94.

Won Lee J, Giraud-Carrier C (2007) Transfer learning in decision trees. *2007 Internat. Joint Conf. Neural Networks* (IEEE, Piscataway, NJ), 726–731.

Ye F, Brodie J, Miles T, Ezzat AA (2023) AIRU-WRF: A physics-guided spatio-temporal wind forecasting model and its application to the U.S. North Atlantic offshore wind energy areas. Preprint, submitted March 3, https://arxiv.org/abs/2303.02246v1.

Copyright of Manufacturing & Service Operations Management (M&SOM) is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.